

## Question 1 (20 points)

### *Partie 1 - Calcul des données*

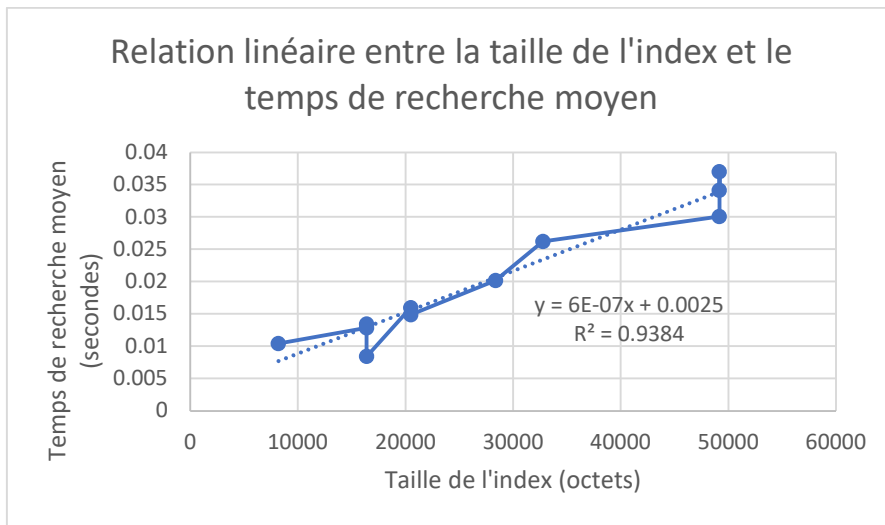
Pour les tests de temps de recherche, on a choisi de tester les 10 mots suivants 100 fois, et de diviser le total de temps obtenu par 1000 afin de calculer le temps de recherche moyen pour toutes les tailles d'index testées.

Mots recherchés : « what », « Shakespeare », « number », « complain », « law », « mention », « appreciate », « work », « school », « object ».

| Nombre de fichiers | Taille totale des fichiers (octets) | Temps d'indexage (secondes) | Taille de l'index (octets) | Temps de recherche moyen (secondes) |
|--------------------|-------------------------------------|-----------------------------|----------------------------|-------------------------------------|
| 10                 | 13 407 003                          | 1.553                       | 8 192                      | 0.010397                            |
| 20                 | 21 913 666                          | 2.432                       | 16 384                     | 0.01285                             |
| 30                 | 65 915 180                          | 2.966                       | 16 384                     | 0.013401                            |
| 50                 | 35 113 013                          | 4.127                       | 16 384                     | 0.008423                            |
| 75                 | 48 605 693                          | 5.768                       | 20 480                     | 0.015959                            |
| 100                | 68 656 443                          | 7.85                        | 20 480                     | 0.014849                            |
| 200                | 165 775 829                         | 24.863                      | 28 372                     | 0.020166                            |
| 300                | 247 287 812                         | 26.324                      | 32 768                     | 0.02621                             |
| 400                | 309 615 653                         | 32.293                      | 49 152                     | 0.03005                             |
| 500                | 384 639 221                         | 44.274                      | 49 152                     | 0.036999                            |
| 594                | 426 138 189                         | 52.283                      | 49 152                     | 0.034139                            |

### *Partie 2 - performance de Lucene selon la taille de l'index*

D'après les temps de recherche calculés dans la partie précédente, on peut en effet affirmer que la vitesse de performance de Lucene augmente avec l'augmentation de la taille de l'index ainsi qu'avec la taille des fichiers qui y sont indexés et le nombre de fichiers indexés. Donc selon toutes les mesures, Lucene est moins performant lorsque l'index est grand. L'estimation d'Excel nous donne l'équation suivante :

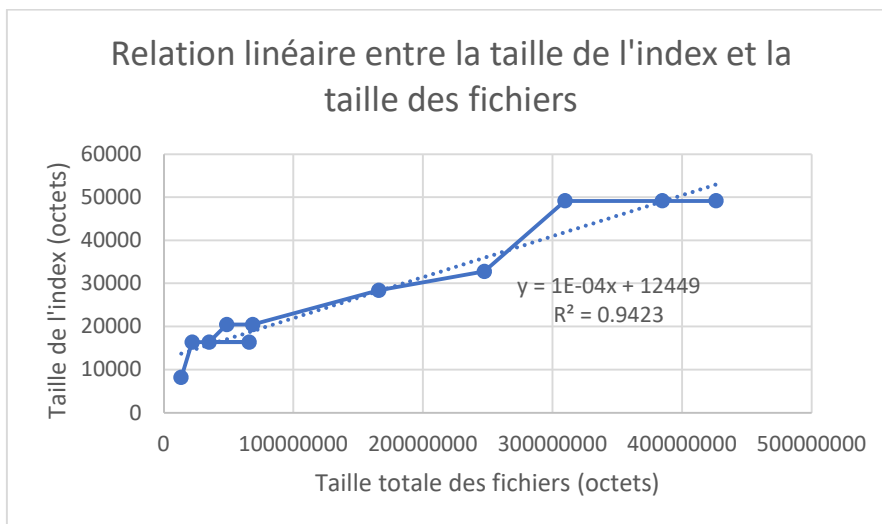
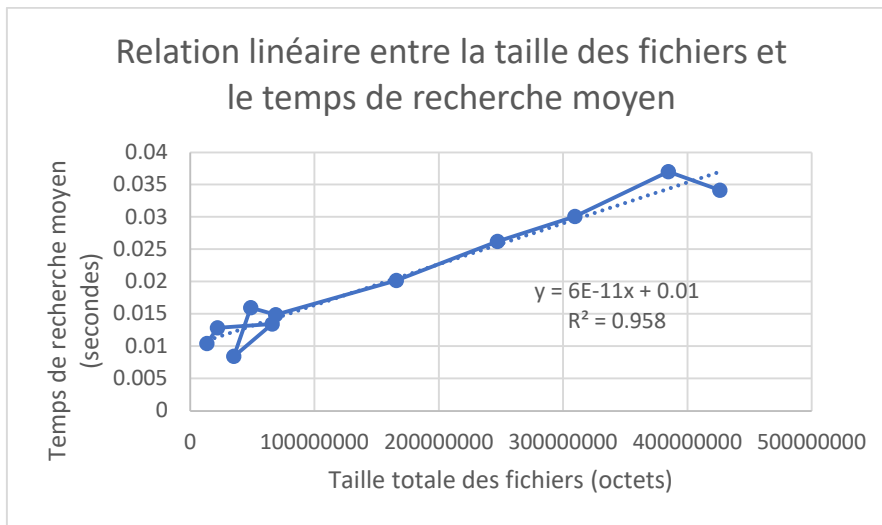


En utilisant cette équation, on peut estimer que si la taille de l'index était 1000 fois plus grande, on aurait un temps de recherche d'environ 29.5 secondes, donc un temps environ 864 plus grand. On peut donc affirmer que si la taille de l'index est multipliée par 1000, le temps de recherche moyen sera environ 1000 fois plus grand aussi.

Ceci peut perdre en exactitude lorsque la taille de l'index devient immense, ce qui n'est pas le cas même en indexant des documents occupant 1000 fois la taille du cédérom du projet Gutenberg.

### *Partie 3 - extrapolation à partir des données calculées*

On peut utiliser Excel afin d'estimer l'équation de la relation entre la taille des fichiers indexés et le temps de recherche tel que :



En utilisant l'équation estimée par Excel, on estime que le temps de recherche deviendrait donc environs 9 600 secondes par recherche, soit 2 heures et 40 minutes. Ceci rendrait l'index complètement inutilisable. La taille de l'index serait d'environ 16Go selon l'estimation de l'équation, ce qui est quand même moins que la taille totale des fichiers bruts de 1600To.

Il ne serait donc pas fonctionnel d'utiliser l'index tel qu'il est configuré ici pour l'indexation d'autant de fichiers si gros. Cependant, il serait possible de mitiger le temps de recherche en modifiant certaines configurations de l'index et de la recherche dans le logiciel.

En effet, on pourrait commencer par limiter les champs stockés, afin de diminuer la quantité d'information à récupérer à chaque *hit* de la recherche. De plus, il serait

plus efficace de limiter le nombre de documents chargés dans les résultats et lequel de leurs champs sont chargé lorsqu'ils sont récupérés. Cette limitation pourrait être implémentée en utilisant des filtres dans la recherche afin de ne chercher qu'un sous-ensemble de l'index au lieu de devoir toujours chercher l'index entier. Dans cette même optique, il serait judicieux de diminuer le nombre de segments dans l'index afin de limiter le nombre de fois que le logiciel doit décharger puis charger un nouveau segment dans la mémoire vive. Finalement, il serait astucieux de ne pas itérer sur l'intégralité des documents récupérés par la recherche, mais plutôt de se limiter au premier 100 à 200 qui sont plus certains de correspondre à la recherche et donc dont les champs sont les plus appropriés à charger. (ASK Infrabot, 2019)

Ces améliorations seraient nécessaires à implémenter si on nous donnait le mandat d'utiliser Lucene pour indexer 8 milliards de documents occupant un total de 16To. De plus, si on ne se limite pas à l'usage de Lucene, on pourrait aussi utiliser des logiciels de gestion de base de données pour améliorer l'efficacité de manipulation de données massive tel que Hadoop.

## Bibliographie

ASK Infrabot. (2019, June 18). *ImproveSearchingSpeed—Apache Lucene (Java)—Apache Software Foundation*.  
<https://cwiki.apache.org/confluence/display/LUCENE/ImproveSearchingSpeed>

## Annexes

### Annexe 1 – Rapport de la console pour chaque test

```
---RAPPORT---  
La fonction set up index a pris 0.116 secondes.  
Le parcours des fichiers a pris 1.43 secondes.  
Le set up de l'index dans main a pris 1.553 secondes.  
On a indexe 10 fichiers. Les fichiers indexés occupent 13407003 octets sur le disque.  
La taille de l'index est de 8192 octets.  
Les recherches ont pris en moyenne 10.397 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.115 secondes.  
Le parcours des fichiers a pris 2.31 secondes.  
Le set up de l'index dans main a pris 2.432 secondes.  
On a indexe 20 fichiers.  
Les fichiers indexés occupent 21913666 octets sur le disque.  
La taille de l'index est de 16384 octets.  
Les recherches ont pris en moyenne 12.85 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.119 secondes.  
Le parcours des fichiers a pris 2.842 secondes.  
Le set up de l'index dans main a pris 2.966 secondes.  
On a indexe 30 fichiers.  
Les fichiers indexés occupent 25915180 octets sur le disque.  
La taille de l'index est de 16384 octets.  
Les recherches ont pris en moyenne 13.401 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.115 secondes.  
Le parcours des fichiers a pris 4.008 secondes.  
Le set up de l'index dans main a pris 4.127 secondes.  
On a indexe 50 fichiers.  
Les fichiers indexés occupent 35113013 octets sur le disque.  
La taille de l'index est de 16384 octets.  
Les recherches ont pris en moyenne 8.423 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.116 secondes.  
Le parcours des fichiers a pris 5.647 secondes.  
Le set up de l'index dans main a pris 5.768 secondes.  
On a indexe 75 fichiers.  
Les fichiers indexés occupent 48605693 octets sur le disque.  
La taille de l'index est de 20480 octets.  
Les recherches ont pris en moyenne 15.959 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.114 secondes.  
Le parcours des fichiers a pris 7.73 secondes.  
Le set up de l'index dans main a pris 7.85 secondes.  
On a indexe 100 fichiers.  
Les fichiers indexés occupent 68656443 octets sur le disque.  
La taille de l'index est de 20480 octets.  
Les recherches ont pris en moyenne 14.849 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.112 secondes.  
Le parcours des fichiers a pris 24.744 secondes.  
Le set up de l'index dans main a pris 24.863 secondes.  
On a indexe 200 fichiers.  
Les fichiers indexes occupent 165775829 octets sur le disque.  
La taille de l'index est de 28672 octets.  
Les recherches ont pris en moyenne 20.166 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.112 secondes.  
Le parcours des fichiers a pris 26.205 secondes.  
Le set up de l'index dans main a pris 26.324 secondes.  
On a indexe 300 fichiers.  
Les fichiers indexes occupent 247287812 octets sur le disque.  
La taille de l'index est de 32768 octets.  
Les recherches ont pris en moyenne 26.21 millisecondes par recherche.
```

```
La fonction set up index a pris 0.113 secondes.  
Le parcours des fichiers a pris 32.175 secondes.  
Le set up de l'index dans main a pris 32.293 secondes.  
On a indexe 400 fichiers.  
Les fichiers indexes occupent 309615653 octets sur le disque.  
La taille de l'index est de 49152 octets.  
Les recherches ont pris en moyenne 30.05 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.115 secondes.  
Le parcours des fichiers a pris 44.153 secondes.  
Le set up de l'index dans main a pris 44.274 secondes.  
On a indexe 500 fichiers.  
Les fichiers indexes occupent 384639221 octets sur le disque.  
La taille de l'index est de 49152 octets.  
Les recherches ont pris en moyenne 36.999 millisecondes par recherche.
```

```
---RAPPORT---  
La fonction set up index a pris 0.114 secondes.  
Le parcours des fichiers a pris 52.163 secondes.  
Le set up de l'index dans main a pris 52.283 secondes.  
On a indexe 594 fichiers.  
Les fichiers indexes occupent 426138189 octets sur le disque.  
La taille de l'index est de 49152 octets.  
Les recherches ont pris en moyenne 34.139 millisecondes par recherche.
```

## Annexe 2 – Exemple de résultat d'une recherche automatisée par le logiciel (fait sur 30 fichiers indexés)

```
Resultat: 27 documents contenant object
ID: Doc 29 - Lucene ID: 29 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 22 - Lucene ID: 22 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 3 - Lucene ID: 2 - Titre: title-3-2cahe10.txt - Auteur: auteur-3-2cahe10.txt
ID: Doc 19 - Lucene ID: 19 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 1 - Lucene ID: 3 - Titre: title-1-1cahe10.txt - Auteur: auteur-1-1cahe10.txt
ID: Doc 23 - Lucene ID: 23 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 13 - Lucene ID: 17 - Titre: title-13-chldh10.txt - Auteur: auteur-13-chldh10.txt
ID: Doc 21 - Lucene ID: 21 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 6 - Lucene ID: 6 - Titre: title-6-8year10.txt - Auteur: auteur-6-8year10.txt
ID: Doc 18 - Lucene ID: 11 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 11 - Lucene ID: 12 - Titre: title-11-btowe10.txt - Auteur: auteur-11-btowe10.txt
ID: Doc 24 - Lucene ID: 24 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 9 - Lucene ID: 4 - Titre: title-9-benhr10.txt - Auteur: auteur-9-benhr10.txt
ID: Doc 8 - Lucene ID: 7 - Titre: title-8-beheb10.txt - Auteur: auteur-8-beheb10.txt
ID: Doc 4 - Lucene ID: 5 - Titre: title-4-2yb4m10.txt - Auteur: auteur-4-2yb4m10.txt
ID: Doc 5 - Lucene ID: 9 - Titre: title-5-8rbaa10.txt - Auteur: auteur-5-8rbaa10.txt
ID: Doc 26 - Lucene ID: 26 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 7 - Lucene ID: 8 - Titre: title-7-andsj10.txt - Auteur: auteur-7-andsj10.txt
ID: Doc 20 - Lucene ID: 20 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 15 - Lucene ID: 13 - Titre: title-15-cstwy11.txt - Auteur: auteur-15-cstwy11.txt
ID: Doc 10 - Lucene ID: 18 - Titre: title-10-bgital0.txt - Auteur: auteur-10-bgital0.txt
ID: Doc 2 - Lucene ID: 1 - Titre: title-2-lvkipl1.txt - Auteur: auteur-2-lvkipl1.txt
ID: Doc 16 - Lucene ID: 14 - Titre: title-16-cyrus10.txt - Auteur: auteur-16-cyrus10.txt
ID: Doc 17 - Lucene ID: 15 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 27 - Lucene ID: 27 - Titre: A Damsel in Distress - Auteur: Pelham Grenville Wodehouse
ID: Doc 12 - Lucene ID: 10 - Titre: title-12-cbtlsl2.txt - Auteur: auteur-12-cbtlsl2.txt
ID: Doc 14 - Lucene ID: 16 - Titre: title-14-cptcr11a.txt - Auteur: auteur-14-cptcr11a.txt
```

## Annexe 3 – Contenu du fichier gutenber2.java

```
import java.io.*;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.Files;
import java.util.stream.Stream;
import java.util.Arrays;
import java.util.Scanner;
import java.util.regex.Pattern;
import java.lang.Integer;

// Utilisation des différentes classes de Lucene
import org.apache.lucene.queryparser.classic.*;
import org.apache.lucene.analysis.standard.*;
import org.apache.lucene.analysis.en.*;
import org.apache.lucene.analysis.*;
import org.apache.lucene.document.*;
import org.apache.lucene.index.*;
import org.apache.lucene.util.*;
import org.apache.lucene.search.*;
import org.apache.lucene.store.*;

public class gutenber2 {
```

```
public static String doc_titre = "";
public static String doc_auteur = "";
public static String doc_contenu = "";
public static String rep_index;
public static Analyzer analyseur;
public static IndexWriterConfig conf;
public static Directory dir;
public static IndexWriter writer;
public static int id = 0;
public static String s_id = "ID";
public static String s_titre = "Titre";
public static String s_auteur = "Auteur";
public static String s_contenu = "Contenu";
public static String report = "---RAPPORT---\n"; //rapport
qui sera afficher dans la console apres les test qui mettra //en
evidence les temps d'operation et les tailles des ficheirs

public static void setupindex(){
    try {
        //set up l'index et l'analyseur pour l'ajout de
documents
        rep_index = ".\\lucene-4.6.1\\etext10\\";
        //rep_index = ".\\lucene-4.6.1\\etext20\\";
        //rep_index = ".\\lucene-4.6.1\\etext30\\";
        //rep_index = ".\\lucene-4.6.1\\etextxx\\";
        analyseur = new
EnglishAnalyzer(Version.LUCENE_46);
        conf = new
IndexWriterConfig(Version.LUCENE_46,analyseur);
        dir = FSDirectory.open(new File(rep_index));
        writer = new IndexWriter(dir, conf);
    } catch (Exception e) {
        System.err.println("Erreur rencontrée dans
setupindex: " + e.toString());
    }
}

public static void addtoindex(File textFile) {
    if (id>=10) return;
    //if (id>=20) return;
    //if (id>=30) return;
    //trouver le titre et l'auteur dans le fichier texte
    try {
        Scanner scanner = new Scanner(textFile);
        while(scanner.hasNext()){
            String line = scanner.nextLine();
            if (line.startsWith("Title: ")){
```

```
        doc_titre = line.substring(7);
    }
    else if (line.startsWith("Author: ")){
        doc_auteur = line.substring(8);
    }
}
} catch (Exception e) {
    System.err.println("Erreur rencontrée dans
addtoindex pendant le scan du fichier texte: " +
e.getMessage());
}
try {
    //fabriquer le document
    Document doc = new Document();
    //id
    doc.add(new StringField(s_id, "Doc "+id,
Field.Store.YES));
    //titre
    if (!doc_titre.equals("")) {
        doc.add(new StringField(s_titre, doc_titre,
Field.Store.YES));
    } else {
        doc.add(new StringField(s_titre, "title-
"+id+"-"+textFile.getName(), Field.Store.YES));
    }
    //auteur
    if (!doc_auteur.equals("")) {
        doc.add(new StringField(s_auteur, doc_auteur,
Field.Store.YES));
    } else {
        doc.add(new StringField(s_auteur, "auteur-
"+id+"-"+textFile.getName(), Field.Store.YES));
    }
    //contenu
    Reader r = new BufferedReader(new
FileReader(textFile));
    org.apache.lucene.document.TextField content = new
org.apache.lucene.document.TextField(s_contenu, r);
    doc.add(content);
    //ajouter le document a l'index
    writer.addDocument(doc);
    writer.commit();
    System.out.println("Document "+(id++)+" ajoute.");
//INCREMENT ID!!! IMPORTANT!!!!
    return;
} catch (Exception e) {
    System.err.println("Erreur rencontrée dans
addtoindex pendant la creation du document: " + e.toString());
}
}
```

```
public static void main(String[] args) {
    long totalFileSize = 0; //taille totale des fichiers
indexer en octets ///////////////
    //ajout a l'index
    long avantSetUpIndexDansMain =
System.currentTimeMillis(); ///////////////
    try{
        long avantFCTsetupindex =
System.currentTimeMillis(); ///////////////
        setupindex();
        long apresFCTsetupindex =
System.currentTimeMillis(); ///////////////
        report += "La fonction set up index a pris
" + (apresFCTsetupindex - avantFCTsetupindex) / 1000.0 + "
secondes.\n"; ///////////////
        Path projetgutenberg = Paths.get(".\\PG2003-
08_files\\pg\\");
        //pour tous les fichiers dans le repertoire du
projet gutenberg
        long avantParcoursFichier =
System.currentTimeMillis(); ///////////////
        for (File dirFile :
projetgutenberg.toFile().listFiles()) {
            //si le fichier est un repertoire et contient
son nom est etext##
            if (id < 10 && dirFile.isDirectory() &&
Pattern.matches(".\\\\\\\\PG2003-08_files\\\\\\\\pg\\\\\\\\etext\\\\d\\\\d",
dirFile.getPath().toString())) {
                //if (id < 20 && dirFile.isDirectory() &&
Pattern.matches(".\\\\\\\\PG2003-08_files\\\\\\\\pg\\\\\\\\etext\\\\d\\\\d",
dirFile.getPath().toString())) {
                    //if (id < 30 && dirFile.isDirectory() &&
Pattern.matches(".\\\\\\\\PG2003-08_files\\\\\\\\pg\\\\\\\\etext\\\\d\\\\d",
dirFile.getPath().toString())) {
                        //if (dirFile.isDirectory() &&
Pattern.matches(".\\\\\\\\PG2003-08_files\\\\\\\\pg\\\\\\\\etext\\\\d\\\\d",
dirFile.getPath().toString())) {
                            //pour tous les fichiers dans le
repertoire etext##
                            for (File textFile : dirFile.listFiles()){
                                //si c'est un fichier texte
                                if (id < 10 &&
textFile.getName().endsWith(".txt")) {
                                    //if (id < 20 &&
textFile.getName().endsWith(".txt")) {
                                        //if (id < 30 &&
textFile.getName().endsWith(".txt")) {
                                            //if
(textFile.getName().endsWith(".txt")) {
```

```
                addtoindex(textFile);
                totalFileSize +=
textFile.length(); ///////////////
            }
            else if (id>=10) break;
            //else if (id>=20) break;
            //else if (id>=30) break;
        }
    }
    else if (id>=10) break;
    //else if (id>=20) break;
    //else if (id>=30) break;
}
    long apresParcoursFichier =
System.currentTimeMillis(); ///////////////
    report += "Le parcours des fichiers a pris
"+(apresParcoursFichier-avantParcoursFichier)/1000.0+
secondes.\n"; ///////////////
    } catch (Exception e) {
        System.err.println("Erreur rencontrée dans main
pendant l'ajout: " + e.toString());
    }
    long apresSetUpIndexDansMain =
System.currentTimeMillis(); ///////////////
    report += "Le set up de l'index dans main a pris
"+(apresSetUpIndexDansMain-avantSetUpIndexDansMain)/1000.0+
secondes.\n"; ///////////////
    report += "On a indexe "+id+" fichiers.\n";
    report += "Les fichiers indexes occupent
"+totalFileSize+" octets sur le disque.\n"; ///////////////
    long indexSize =
Paths.get(rep_index).toFile().length(); //taille de l'index en
octets ///////////////
    report += "La taille de l'index est de "+indexSize+
octets.\n"; ///////////////
    //recherche dans l'index
    try {
        String[] champs_recherche = {s_titre, s_auteur,
s_contenu};
        QueryParser parser = new
MultiFieldQueryParser(Version.LUCENE_46, champs_recherche,
analyseur);
        long totalTime = 0; //temps total passe a
rechercher dans l'index en millisecondes
        // Boucle permettant de faire les 100 recherches
automatiquement
        for (int j=0; j<100; j++) {
            String[] recherches = {"what", "Shakespeare",
"number", "complain", "law", "mention", "appreciate", "work",
"school", "object"};
```

```
        //faire une recherche pour chaque mot
        for (String lettres : recherches){
            long debutRecherche =
System.currentTimeMillis(); ///////////////
            // lancement de la recherche
            Query query = parser.parse(lettres);
            //Query query =
MultiFieldQueryParser.parse(lettres, champs_recherche,
analyseur);
                IndexSearcher searcher = new
IndexSearcher(DirectoryReader.open(FSDirectory.open(new
File(rep_index))));
                TopDocs results =
searcher.search(query,4096);
                ScoreDoc[] hits = results.scoreDocs;

                // Impression des résultats
                System.out.println("Resultat: " +
hits.length + " documents contenaient " + lettres);
                for(int i = 0; i < hits.length; i++){
                    Document doc =
searcher.doc(hits[i].doc);
                    int lucene_id = hits[i].doc;
                    String my_id = doc.get(s_id);
                    String titre = doc.get(s_titre);
                    String auteur = doc.get(s_auteur);
                    System.out.println("ID: " + my_id + "
- Lucene ID: " + lucene_id + " - Titre: " + titre + " -
Auteur: " + auteur );
                }
                long finRecherche =
System.currentTimeMillis(); ///////////////
                totalTime += (finRecherche-
debutRecherche); //ajout le temps de cette recherche au total
//////////////////
            }
        }
        report += "Les recherches ont pris en moyenne
" + (totalTime)/(10*100.0) + " millisecondes par recherche.\n";
//////////////////
    } catch (Exception e) {
        System.err.println("Erreur rencontrée dans main
pendant la recherche: " + e.toString());
    }
    System.out.println(report); ///////////////
}
}
```